Software Engineering — Lab 5

Static Code Analysis

Name: Sampreeth Jaladi

SRN: PES1UG24AM246

Section: E · PES University, RR Campus

Repository: L5_08_warehouse_assets (`maintenance_system.py`)

Date: 02 November 2026

1. Objective

Use Pylint, Bandit and Flake8 to find and fix quality, security and style issues in `maintenance_system.py`, a maintenance registry for warehouse equipment. It registers machines, records service dates, raises alerts for overdue inspections and exports a JSON report.

2. Setup and Commands

```
python3 -m venv venv && source venv/bin/activate
pip install pylint bandit flake8
python maintenance_system.py
pylint maintenance_system.py > pylint_report.txt
bandit -r maintenance_system.py > bandit_report.txt
flake8 maintenance_system.py > flake8_report.txt
```

3. Results Summary

Tool	Before	After
Pylint	6.55/10 (19 messages)	**10.00/10**
Bandit	2 issues (1 Medium, 1 Low)	**0 issues**
Flake8	12 violations	**0 violations**

The full before and after reports are in Appendix A and in `reports/before/` and `reports/after/` in the repository.

4. Issues Identified and Fixed

Line numbers refer to the original starter file.

#	Issue	Tool / Code	Severity	Line(s)	Fix Approach
1	Insecure temp file (`tempfile.mktemp`)	Bandit B306	Medium	40	`NamedTemporaryFile(delete=False)` creates the file atomically
2	`assert` used as a runtime check	Bandit B101	Low	61	Explicit `if not assets:` check
3	Mutable default argument `logs=[]`	Pylint W0102	Medium	10	Default `None`; explicit module-level `registration_log`
4	Bare except	Pylint W0702, Flake8 E722	Medium	18	Catch `KeyError` / `ValueError` only

#	Issue	Tool / Code	Severity	Line(s)	Fix Approach
5	`open()` without `with` or encoding	Pylint R1732, W1514	Medium	34, 41	`with open(..., encoding="utf-8")` + `json.dump`
6	`global` statement	Pylint W0603	Medium	30	Interval stored in a `settings` dict
7	Unused import `logging`	Pylint W0611, Flake8 F401	Low	2	Removed
8	`printAlerts` not snake_case	Pylint C0103	Low	46	Renamed to `print_alerts`
9	Indexing dict inside key loop	Pylint C0206	Low	23	`assets.items()`
10	Missing docstrings	Pylint C0114, C0116	Low	all	Module and function docstrings added
11	Blank lines / long line	Flake8 E302, E305, E501	Low	10–66	PEP 8 spacing; wrapped line 11
12	Code runs on import	Manual	Low	66	`if __name__ == "__main__":` guard

Logic bugs found by running the program (not flagged by any tool)

#	Bug	Line	Observed	Fix
13	Empty asset ID accepted	53	Asset `""` registered and alerted	Reject empty or whitespace-only IDs
14	Duplicate ID overwrites	54	FL-01 Forklift became “Crane”	Reject IDs that are already registered
15	Negative interval accepted	57	`-30` days made every asset overdue	Accept positive integers only
16	Unvalidated date input	58–60	Input bad crashed `strptime` with `ValueError`	Validate format; reject future dates

5. Key Fixes (Before → After)

Insecure temp file (B306)

```
        # before
        path = tempfile.mktemp(suffix=".json")
        f = open(path, "w")
        # after
        with tempfile.NamedTemporaryFile(
            "w", suffix=".json", delete=False, encoding="utf-8"
        ) as scratch:
            json.dump(assets, scratch)
```

Mutable default (W0102)

```
        # before
        def register_asset(asset_id, machine_type, logs=[]):
        # after
        def register_asset(asset_id, machine_type, logs=None):
            if logs is None:
                logs = registration_log
```

Bare except (W0702 / E722)

```
        # before
        except:
            print("Could not update", asset_id)
        # after
        except KeyError:
            print("Could not update", asset_id, "- unknown asset")
            return False
```

Global statement (W0603)

```
        # before
        global SERVICE_INTERVAL_DAYS
        SERVICE_INTERVAL_DAYS = days
        # after
        if not isinstance(days, int) or days <= 0:
            return False
        settings["service_interval_days"] = days
```

6. Git Commits

Commit	Purpose
Add Lab 5 starter code	Original `maintenance_system.py`
Add baseline pylint, bandit and flake8 reports	Before-fix reports
Fix Bandit issues	B306 `mktemp`, B101 `assert`
Fix Pylint issues	W0102, W0702, W0603, R1732, W1514, C0103, C0206, docstrings, input validation
Fix Flake8 issues	E302 / E305 blank lines

Commit	Purpose
Add after-fix reports, issue table and reflection	After reports, `issues.md`, `reflection.md`

7. Program Output

Before fixes (Enter pressed at the prompt):

```
Registered FL-01 - total registrations this run: 1
Registered CV-07 - total registrations this run: 2
Registered  - total registrations this run: 3
Registered FL-01 - total registrations this run: 4
Could not update XX-99
Last service date for CV-07 (YYYY-MM-DD, Enter to skip):
ALERT: inspection overdue for FL-01 Crane
ALERT: inspection overdue for CV-07 Conveyor
ALERT: inspection overdue for  Pallet Jack
Report written to maintenance.json
```

After fixes (input `bad` at the prompt):

```
Registered FL-01 - total registrations this run: 1
Registered CV-07 - total registrations this run: 2
Rejected: asset ID must not be empty
Rejected: FL-01 is already registered as Forklift
Could not update XX-99 - unknown asset
Rejected: service interval must be a positive integer, keeping 90 days
Last service date for CV-07 (YYYY-MM-DD, Enter to skip): bad
Invalid date 'bad' - expected YYYY-MM-DD
ALERT: inspection overdue for FL-01 Forklift
Report written to maintenance.json
```

8. Reflection

1. Which issues were the easiest to fix, and which were the hardest?

The easiest were the mechanical style issues: Flake8 blank lines, the unused import, the naming fix and docstrings. Each has one obvious fix that doesn't change behaviour. The hardest needed a design decision. Removing `global` meant choosing where the interval should live. The mutable default couldn't simply become `None` without breaking the running registration count, so that shared state had to become explicit. The logic bugs were hardest of all, because no tool flagged them and I only found them by running the program.

2. Did the tools report any false positives?

None were wrong, but some were context-dependent. Bandit B101 is a false positive in test files, where `assert` is correct; here it was valid, because the guard disappears under `python -O`. Docstring warnings on tiny functions are arguably noise. More important were the false negatives: all three tools passed code that accepted empty IDs, overwrote assets and allowed a negative interval.

3. How would you integrate static analysis into your workflow?

Editor linting on save, plus a `pre-commit` hook running Flake8, `pylint --fail-under=9.5` and Bandit. In CI, a GitHub Actions job on every push and pull request fails the build on any Bandit Medium/High finding, any Flake8 error, or a Pylint score under the threshold. Shared rules live in `pyproject.toml` / `.flake8`, and any suppression (`# nosec`) must carry a written reason.

4. What improvements did you observe?

Pylint rose from 6.55 to 10.00, and Bandit and Flake8 went to zero. The temp file is now created securely, file handles always close, and only expected exceptions are caught. Invalid input is rejected instead of crashing or producing wrong alerts. The code follows PEP 8 and is importable without side effects, which makes it testable.

9. Conclusion

Static analysis quickly surfaced security, reliability and style problems and gave a measurable before/after comparison. It did not catch logic errors, though, so running and testing the program is still necessary alongside the tools.

Appendix A — Static Analysis Reports

pylint_report.txt (before)

```
************* Module maintenance_system
maintenance_system.py:1:0: C0114: Missing module docstring (missing-module-docstring)
maintenance_system.py:10:0: C0116: Missing function or method docstring (missing-function-do
cstring)
maintenance_system.py:10:0: W0102: Dangerous default value [] as argument (dangerous-default
-value)
maintenance_system.py:15:0: C0116: Missing function or method docstring (missing-function-do
cstring)
maintenance_system.py:18:4: W0702: No exception type(s) specified (bare-except)
maintenance_system.py:21:0: C0116: Missing function or method docstring (missing-function-do
cstring)
maintenance_system.py:23:4: C0206: Consider iterating with .items() (consider-using-dict-ite
ms)
maintenance_system.py:29:0: C0116: Missing function or method docstring (missing-function-do
cstring)
maintenance_system.py:30:4: W0603: Using the global statement (global-statement)
maintenance_system.py:33:0: C0116: Missing function or method docstring (missing-function-do
cstring)
maintenance_system.py:34:8: W1514: Using open without explicitly specifying an encoding (uns
pecified-encoding)
maintenance_system.py:34:8: R1732: Consider using 'with' for resource-allocating operations
(consider-using-with)
maintenance_system.py:39:0: C0116: Missing function or method docstring (missing-function-do
cstring)
maintenance_system.py:41:8: W1514: Using open without explicitly specifying an encoding (uns
pecified-encoding)
maintenance_system.py:41:8: R1732: Consider using 'with' for resource-allocating operations
(consider-using-with)
maintenance_system.py:46:0: C0116: Missing function or method docstring (missing-function-do
cstring)
maintenance_system.py:46:0: C0103: Function name "printAlerts" doesn't conform to snake_case
naming style (invalid-name)
maintenance_system.py:50:0: C0116: Missing function or method docstring (missing-function-do
cstring)
maintenance_system.py:2:0: W0611: Unused import logging (unused-import)

-----------------------------------
Your code has been rated at 6.55/10
```

bandit_report.txt (before)

```
Run started:2026-11-02 05:12:00.686185+00:00

Test results:
>> Issue: [B306:blacklist] Use of insecure and deprecated function (mktemp).
   Severity: Medium   Confidence: High
   CWE: CWE-377 (https://cwe.mitre.org/data/definitions/377.html)
   More Info: https://bandit.readthedocs.io/en/1.9.4/blacklists/blacklist_calls.html#b306-mk
temp-q
```

```
   Location: ./maintenance_system.py:40:11
39  def write_scratch_copy():
40      path = tempfile.mktemp(suffix=".json")
41      f = open(path, "w")

--------------------------------------------------
>> Issue: [B101:assert_used] Use of assert detected. The enclosed code will be removed when
compiling to optimised byte code.
   Severity: Low   Confidence: High
   CWE: CWE-703 (https://cwe.mitre.org/data/definitions/703.html)
   More Info: https://bandit.readthedocs.io/en/1.9.4/plugins/b101_assert_used.html
   Location: ./maintenance_system.py:61:4
60          update_service_date("CV-07", d)
61      assert len(assets) > 0, "no assets"
62      printAlerts()

--------------------------------------------------

Code scanned:
    Total lines of code: 55
    Total lines skipped (#nosec): 0
    Total potential issues skipped due to specifically being disabled (e.g., #nosec BXXX): 0

Run metrics:
    Total issues (by severity):
        Undefined: 0
        Low: 1
        Medium: 1
        High: 0
    Total issues (by confidence):
        Undefined: 0
        Low: 0
        Medium: 0
        High: 2
Files skipped (0):
```

flake8_report.txt (before)

```
maintenance_system.py:2:1: F401 'logging' imported but unused
maintenance_system.py:10:1: E302 expected 2 blank lines, found 1
maintenance_system.py:11:80: E501 line too long (98 > 79 characters)
maintenance_system.py:15:1: E302 expected 2 blank lines, found 1
maintenance_system.py:18:5: E722 do not use bare 'except'
maintenance_system.py:21:1: E302 expected 2 blank lines, found 1
maintenance_system.py:29:1: E302 expected 2 blank lines, found 1
maintenance_system.py:33:1: E302 expected 2 blank lines, found 1
maintenance_system.py:39:1: E302 expected 2 blank lines, found 1
maintenance_system.py:46:1: E302 expected 2 blank lines, found 1
maintenance_system.py:50:1: E302 expected 2 blank lines, found 1
maintenance_system.py:66:1: E305 expected 2 blank lines after class or function definition,
found 1
```

pylint_report.txt (after)

```
------------------------------------------------------------------
Your code has been rated at 10.00/10 (previous run: 10.00/10, +0.00)
```

bandit_report.txt (after)

```
Run started:2026-11-02 05:12:57.391759+00:00

Test results:
    No issues identified.

Code scanned:
    Total lines of code: 100
    Total lines skipped (#nosec): 0
    Total potential issues skipped due to specifically being disabled (e.g., #nosec BXXX): 0

Run metrics:
    Total issues (by severity):
        Undefined: 0
        Low: 0
        Medium: 0
        High: 0
    Total issues (by confidence):
        Undefined: 0
        Low: 0
        Medium: 0
        High: 0
Files skipped (0):
```

flake8_report.txt (after)

```
(empty — no violations reported)
```

Appendix B — Final maintenance_system.py

```
        """Warehouse asset maintenance registry.

        Registers machines, records service dates, raises alerts for overdue
        inspections and exports a report to ``maintenance.json``.
        """

        import json
        import tempfile
        from datetime import datetime, timedelta

        DATE_FORMAT = "%Y-%m-%d"

        # Global state
        assets = {}
        registration_log = []
        settings = {"service_interval_days": 90}


        def register_asset(asset_id, machine_type, logs=None):
            """Register an asset; reject empty or duplicate IDs."""
            if logs is None:
                logs = registration_log
            if not asset_id or not asset_id.strip():
                print("Rejected: asset ID must not be empty")
                return False
            if asset_id in assets:
                print("Rejected:", asset_id, "is already registered as",
                      assets[asset_id]["type"])
                return False
            assets[asset_id] = {
                "type": machine_type,
                "last_service": datetime.now().strftime(DATE_FORMAT),
            }
            logs.append(asset_id)
            print("Registered", asset_id,
                  "- total registrations this run:", len(logs))
            return True


        def update_service_date(asset_id, date):
            """Set the last service date (YYYY-MM-DD, not in the future)."""
            try:
                parsed = datetime.strptime(date, DATE_FORMAT)
            except ValueError:
                print("Invalid date", repr(date), "- expected YYYY-MM-DD")
                return False
            if parsed > datetime.now():
                print("Rejected: service date", date, "is in the future")
                return False
            try:
                assets[asset_id]["last_service"] = date
            except KeyError:
                print("Could not update", asset_id, "- unknown asset")
```

```
            return False
        return True


    def due_for_inspection():
        """Return IDs of assets whose last service exceeds the interval."""
        interval = timedelta(days=settings["service_interval_days"])
        due = []
        for asset_id, info in assets.items():
            last = datetime.strptime(info["last_service"], DATE_FORMAT)
            if datetime.now() - last > interval:
                due.append(asset_id)
        return due


    def change_interval(days):
        """Change the service interval; must be a positive whole number."""
        if not isinstance(days, int) or days <= 0:
            print("Rejected: service interval must be a positive integer,",
                  "keeping", settings["service_interval_days"], "days")
            return False
        settings["service_interval_days"] = days
        return True


    def export_service_report(filename="maintenance.json"):
        """Write all assets to a JSON report file."""
        with open(filename, "w", encoding="utf-8") as report:
            json.dump(assets, report, indent=2)
        print("Report written to " + filename)


    def write_scratch_copy():
        """Write a scratch copy of the registry to a secure temp file."""
        with tempfile.NamedTemporaryFile(
            "w", suffix=".json", delete=False, encoding="utf-8"
        ) as scratch:
            json.dump(assets, scratch)
        print("Scratch copy at", scratch.name)


    def print_alerts():
        """Print an alert for every asset that is overdue for inspection."""
        for asset_id in due_for_inspection():
            print("ALERT: inspection overdue for", asset_id,
                  assets[asset_id]["type"])


    def main():
        """Run the demo workflow."""
        register_asset("FL-01", "Forklift")
        register_asset("CV-07", "Conveyor")
        register_asset("", "Pallet Jack")
        register_asset("FL-01", "Crane")
        update_service_date("FL-01", "2025-01-15")
        update_service_date("XX-99", "2025-02-01")
        change_interval(-30)
        date = input("Last service date for CV-07 (YYYY-MM-DD, Enter to skip): ")
```

```
            if date:
                update_service_date("CV-07", date.strip())
            if not assets:
                print("No assets registered; nothing to report")
                return
            print_alerts()
            export_service_report()
            write_scratch_copy()


        if __name__ == "__main__":
            main()
```